CloudLab

Windows Installation Guide

Installing and running CloudLab on a personal Windows PC
via WSL2 (Ubuntu) + Docker Desktop
Generated: September 08, 2026

WSL2 + Ubuntu | Docker Desktop | Postgres 16 | Redis | Go | Node.js 20+

Why WSL2? CloudLab provisions real infrastructure using Linux-only mechanisms — real `iptables` rules for firewalls/NAT/VPC peering, a real dedicated PostgreSQL server per managed database, and a real Docker daemon. None of that exists natively on Windows, so CloudLab runs inside a lightweight Linux environment (WSL2) that Windows hosts directly — no separate physical machine or full virtual machine needed.

1 Before you start

This guide assumes a normal Windows 10 (version 2004+) or Windows 11 PC with an Intel or AMD processor (the common case — see the note on checking this in Part B). You'll go back and forth between two places throughout this guide:

TAG	MEANING
WINDOWS	Run this in a regular Windows PowerShell window, or by clicking things in a Windows app (Docker Desktop's settings, an installer, etc.)
UBUNTU	Run this inside the Ubuntu terminal window (opened from the Start menu after WSL2/Ubuntu is installed) — this is where CloudLab itself actually runs.

> Every restart mentioned in this guide is a normal Windows restart or a lightweight WSL/Docker-engine restart — nothing here is destructive to your files, installed programs, or Windows itself. Where a step says "reboot," it means the same thing as installing a Windows update: close your other work, restart, and continue when it's back.

2 Part A — Install WSL2 and Ubuntu

2.1 Enable WSL2 and install Ubuntu WINDOWS

Open **PowerShell as Administrator** (right-click the Start button → "Terminal (Admin)" or "Windows PowerShell (Admin)") and run:

```
wsl --install -d Ubuntu
```

This enables the underlying Windows virtualization features and begins installing Ubuntu. The very first time you run this, it will typically only get as far as enabling those features and then say it needs a restart — that's expected, not an error.

2.2 Reboot WINDOWS

Restart your PC normally (Start menu → Power → Restart, or run `Restart-Computer` from the same PowerShell window). This is required once, to activate WSL2's virtualization feature — it will not affect your files or other programs.

2.3 Finish installing Ubuntu WINDOWS

After the reboot, open PowerShell as Administrator again and run the same command a second time:

```
wsl --install -d Ubuntu
```

This time the virtualization features are active, so it will actually download and install Ubuntu (a few minutes). Confirm it succeeded:

```
wsl -l -v
```

You should see `Ubuntu` listed with `VERSION` = `2`.

2.4 Create your Linux account UBUNTU

Ubuntu will launch automatically and ask you to create a **default Unix user account** — this is a Linux username/password, completely separate from your Windows login:

- **Username**: any lowercase name with no spaces (e.g. `cloudlab`, or whatever it suggests).
- **Password**: any password you choose — it does not need to match your Windows password. Remember it, since you'll need it for every `sudo` command below.

> **Note:** when you type the password, nothing appears on screen at all — no dots, no cursor movement. That's normal Linux terminal behavior, not a bug. Type carefully and press Enter.

Once done, you'll land at a prompt like `yourname@DESKTOP-XXXX:~$` — you're now inside Ubuntu. Confirm sudo works:

```
sudo apt-get update
```

3 Part B — Install Docker Desktop

3.1 Check your CPU architecture WINDOWS

Almost all Windows PCs are **AMD64** (Intel or AMD processors) — ARM64 only applies to a small number of "Windows on ARM" devices (Surface Pro X, some Copilot+ laptops). To confirm, run in PowerShell:

```
systeminfo | findstr /B /C:"System Type"
```

`x64-based PC` → download **AMD64**. `ARM64-based PC` → download **ARM64**. If unsure, AMD64 is correct for the overwhelming majority of machines.

3.2 Download and install Docker Desktop WINDOWS

1. Download the installer for your architecture from `docker.com/products/docker-desktop`.
2. Double-click the downloaded `Docker Desktop Installer.exe`.
3. Leave **"Use WSL 2 instead of Hyper-V"** checked (default).
4. When asked **"Install for me only" vs "all users"**: choose **"Install for me only"** (the recommended default) unless multiple Windows logins on this PC need it.
5. When asked about **Windows containers**: say **No / leave unchecked**. CloudLab needs Linux containers (the default) — Windows containers are a different, unrelated feature for legacy Windows-only apps.
6. Click Install/OK and wait. The "Extracting manifest" step can genuinely take several minutes, especially with antivirus scanning each file — see the troubleshooting table in §7 if it seems frozen.
7. Reboot when prompted (a normal restart).

3.3 First launch WINDOWS

Docker Desktop should open automatically after the reboot (or launch it from the Start menu). Accept the service agreement. When asked to sign in, look for a smaller **"Skip for now"** / **"Continue without signing in"** option — a Docker account isn't needed for anything CloudLab does. Wait until the whale icon in the system tray stops animating.

3.4 Enable WSL integration WINDOWS

1. In Docker Desktop, click the gear icon (**Settings**) top right.
2. Go to **Resources → WSL Integration**.
3. Turn the toggle for **Ubuntu ON**.
4. Click **Apply & Restart** (this restarts Docker's engine only — a few seconds, not a full PC reboot).

3.5 Verify from Ubuntu UBUNTU

Open a fresh Ubuntu terminal (Start menu → "Ubuntu") and run:

```
docker run hello-world
```

Expected output:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

> **If you instead see** `permission denied while trying to connect to the docker API at unix:///var/run/docker.sock`: this is a common first-time hiccup, not a real problem — see §7's troubleshooting table for the exact fix (a WSL restart almost always resolves it).

4 Part C — Install the remaining prerequisites

All commands in this section run inside **UBUNTU**.

4.1 Node.js 20+

Ubuntu's built-in `nodejs` package is usually too old for this project, so install it from NodeSource's official repository instead:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Confirm: `node --version` should print `v20.x.x` or higher.

4.2 PostgreSQL 16, Redis, Go, and networking tools

```
sudo apt-get install -y postgresql-16 redis-server golang busybox-static iptables
```

Confirm each installed:

```
psql --version
redis-cli --version
go version
```

Each should print a real version number.

5 Part D — Get the CloudLab source and configure it

All commands in this section run inside **UBUNTU**.

5.1 Unzip the source into your Linux home directory

Put it under your Ubuntu home directory (~), not the Windows C: drive (`/mnt/c/...`) — file access across that boundary is much slower and can cause odd Docker permission issues.

```
cd ~
unzip /mnt/c/Users/<your-windows-username>/Downloads/CloudLab_Source_Complete_v3.zip -d cloudlab
cd cloudlab
npm install
```

5.2 Configure environment variables

```
cp .env.example .env
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

Paste that command's output into `.env` as the value of `SECRETS_MASTER_KEY=` (open it with `nano .env`, edit the line, then Ctrl+O, Enter, Ctrl+X to save and exit).

5.3 Set up Postgres for CloudLab

```
sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'cloudlab';"
sudo -u postgres createdb cloudlab
npm run db:migrate
```

> **Already have pgAdmin on Windows?** WSL2 forwards `localhost` to Windows automatically, so you can register a server in pgAdmin pointing at host `localhost`, port 5432, user `postgres`, password `cloudlab` — you'll see the `cloudlab` database appear there once migrations run, handy for browsing tables visually.

The app also provisions **real, separate Postgres servers on demand** (one per managed-database or BigQuery-dataset resource you create in the app), which needs passwordless sudo access to the `postgres` OS account:

```
echo "$(whoami) ALL=(postgres) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/cloudlab-worker
```

5.4 Build the local container images

Nothing is pulled from a registry here — every image is compiled and built from the included source:

```
bash infra/guest-image/build.sh
bash infra/dns-image/build.sh
bash infra/lb-image/build.sh
bash infra/registry-image/build.sh
bash infra/bucket-image/build.sh
```

6 Part E — Run and verify CloudLab

All commands in this section run inside `UBUNTU`, from the `~/cloudlab` directory.

6.1 Start the two long-running processes

Open two separate Ubuntu terminal windows.

```
# terminal 1
npm run dev:api
```

```
# terminal 2 -- must run with root privileges: it calls real iptables/sysctl
# commands directly for firewall rules, NAT gateways, and VPC peering
sudo -E npm run dev:worker
```

6.2 Open the console

In your normal Windows browser, go to:

```
http://localhost:4000
```

WSL2 forwards this port to Windows automatically — no extra configuration needed. Register your first account and you're in.

6.3 Run the automated checks (optional but recommended)

```
npm test        # 20 checks against the API layer
npm run e2e       # 31 checks against the full running stack
```

Both should finish reporting all checks passed — a good sign your install is fully healthy end to end.

7 Troubleshooting reference

Real issues that came up while working through this exact install, and their fixes:

SYMPTOM	FIX
`wsl -l -v` says "no installed distributions" right after `wsl --install`	Expected the first time — the command only enabled Windows features so far. Reboot, then run `wsl --install -d Ubuntu` again.
Docker Desktop installer stuck at "Extracting manifest"	Check Task Manager for the installer's CPU/Disk activity — if non-zero, it's just slow (antivirus scanning), wait 5–10 min. If truly 0% for a long time: End task, temporarily disable antivirus real-time protection, re-download the installer, run it as Administrator, retry.
Docker Desktop asks to sign in on first launch	Look for a smaller "Skip for now" link below the main sign-in button. Not required for CloudLab either way.
`docker run hello-world` → "permission denied ... docker.sock"	Close all Ubuntu windows → run `wsl --shutdown` in PowerShell → confirm Docker Desktop's WSL Integration toggle for Ubuntu is still on (toggle off/on and Apply & Restart if unsure) → reopen Ubuntu → retry. If it persists: `sudo usermod -aG docker $USER`, then repeat the `wsl --shutdown` + reopen step.
`node --version` shows below v20 after `apt-get install nodejs`	Use the NodeSource setup script in §4.1 instead of the plain `apt-get install nodejs` — Ubuntu's default repo version is usually too old.
Worker process errors touching `iptables`/`sysctl`	The worker must be started with `sudo -E npm run dev:worker`, not a plain `npm run dev:worker` — it needs root privileges for real firewall/NAT/peering rules.
Postgres-backed features (managed databases, BigQuery datasets) fail to provision	Confirm the sudoers line from §5.3 was added correctly: `cat /etc/sudoers.d/cloudlab-worker` should show your username with NOPASSWD access to the postgres account.

8 Quick-reference checklist

	STEP
☐	WSL2 + Ubuntu installed, Linux user account created
☐	Docker Desktop installed (correct CPU architecture, Linux containers, WSL integration enabled)
☐	`docker run hello-world` succeeds from Ubuntu
☐	Node.js 20+, PostgreSQL 16, Redis, Go, busybox-static, iptables installed
☐	CloudLab source unzipped under `~/cloudlab` (not `/mnt/c/...`), `npm install` run
☐	`.env` configured with a generated `SECRETS_MASTER_KEY`
☐	Postgres role password set, `cloudlab` database created, migrations run, sudoers NOPASSWD entry added
☐	All 5 local container images built (`infra/*/build.sh`)
☐	`npm run dev:api` and `sudo -E npm run dev:worker` both running
☐	`http://localhost:4000` loads in your Windows browser, account registered
☐	`npm test` and `npm run e2e` both pass

This guide reflects a real, working WSL2 + Docker Desktop installation path for a personal Windows machine with an Intel/AMD (AMD64) processor. Every command and troubleshooting fix here was verified against the actual CloudLab source and a real install session.